

Intelligent GST Reconciliation Using Knowledge Graphs

Comprehensive Project Architecture, Machine Learning & Technical Report

Author / Engineer:	Karthikeya (Full-Stack & ML Engineer)	Date:	September 2026
Frontend Stack:	React 19 + Vite + Framer Motion	Backend Stack:	FastAPI + Python 3.11
Primary Database:	MongoDB Atlas (System of Record)	Graph Database:	Neo4j 5 Community / Enterprise
ML Engine:	RandomForestClassifier (300 trees)	Status:	Production Ready / Verified

1. Executive Summary & Problem Statement

India's Goods and Services Tax (GST) system requires businesses to rigorously reconcile invoices across multiple monthly returns (GSTR-1, GSTR-2B, GSTR-3B) before claiming **Input Tax Credit (ITC)**. Discrepancies lead to blocked credit worth crores of rupees, legal penalties under Section 73/74 of the CGST Act, and severe cash flow disruptions.

Why Traditional SQL Systems Fail:

- Single-hop Table Limits:** Flat SQL table-matching verifies if an invoice exists in GSTR-1, but cannot traverse to the supplier's GSTR-3B return to confirm if tax was remitted.
- Opaque Audit Trails:** Traditional software flags an invoice as 'Unmatched' without providing statutory evidence or legal citations required during tax audits.
- Reactive Risk Management:** Audits happen months after filing when tax credits are already lost, rather than predicting vendor non-compliance risk proactively.

The GST ReconcileAI Solution: This application models the complete GST supply chain as a **Knowledge Graph** (Neo4j), detects multi-hop discrepancies via Cypher graph traversal, predicts vendor risk using a **Random Forest classifier**, and generates **legally grounded explainable audit trails**.

2. System Architecture & Design

The application is built on a 4-tier modular architecture designed for high availability, sub-second query performance, and graceful degradation when underlying services are offline.

Layer	Technology Stack	Core Responsibilities
Frontend	React 19, Vite, Framer Motion, Chart.js, react-force-graph-2d	Single-page interactive UI, real-time KPI dashboards, interactive force graph, filterable reconciliation tables, audit cards.

State Layer	React Context API (DataContext, AuthContext, ThemeContext)	Central state, live telemetry polling, API dispatching, client-side fallback graph projection when Neo4j is down.
Backend API	FastAPI, Python 3.11, PyMongo, Uvicorn	High-performance async REST API, CORS middleware, MongoDB data access, model serving, auth security.
Database Layer	MongoDB Atlas (Primary) + Neo4j 5 (Knowledge Graph)	MongoDB serves as system of record; Neo4j maintains graph projection of Vendors, Invoices, Returns, IRNs, e-Way Bills.
Machine Learning	Scikit-Learn RandomForestClassifier (300 trees)	Predicts vendor compliance risk from 8 graph-derived features (held-out accuracy 76.8%, ROC-AUC 0.853).

3. Dual-Database Storage & Entity Schema

To combine document flexibility with graph traversal speed, the system uses a dual-database pattern:

A. MongoDB Collections (System of Record)

- **vendors:** Stores vendor profiles, state, risk score, transaction count, missed filings, average days late.
- **invoices:** Stores taxable amount, CGST/SGST/IGST breakdown, total tax, HSN code, filing period, status flags.
- **taxpayer:** Stores entity details for the buyer claiming credit (e.g., Quadric Manufacturing Pvt Ltd).
- **returns:** Stores per-vendor, per-period GSTR-3B filing records (tracks actual tax payment).
- **users & alerts:** User authentication accounts (bcrypt hashed) and real-time alert logs.

B. Neo4j Knowledge Graph Schema

Data from MongoDB is projected into Neo4j via graph_sync.py into 7 Node types and 8 Relationship types:

Entity (Node Label)	Properties Stored	Relationships (Edges)
(:Taxpayer)	gstin, name, state, legal_name	-[:RECORDED_IN_PR]-> (:Invoice) -[:RECEIVES]-> (:GSTR {type:'GSTR-2B'})
(:Vendor)	gstin, name, state, risk_score, status, pagerank	-[:ISSUED_INVOICE]-> (:Invoice) -[:FILED_RETURN]-> (:GSTR3B)
(:Invoice)	id, taxable_amount, total_tax, hsn, period, match_status	-[:REPORTED_IN]-> (:GSTR) -[:ELECTRONIC_VERSION]-> (:EInvoice) -[:COVERS_SHIPMENT]-> (:EWayBill)
(:GSTR)	type ('GSTR-1', 'GSTR-2B'), period	Target of :REPORTED_IN from invoices.
(:GSTR3B)	gstin, period, filed (boolean), status	Target of :FILED_RETURN from vendors.
(:EInvoice)	irn	Target of :ELECTRONIC_VERSION.
(:EWayBill)	id	Target of :COVERS_SHIPMENT.

4. Graph-Traversal Reconciliation Engine

The reconciliation engine (reconcile.py) executes Cypher graph traversals across 7 distinct mismatch classes. It distinguishes between **Structural Checks** (derived from absent graph edges) and **Field-Level Checks**:

Key Cypher Reconciliation Patterns:

```
// 1. Missing in GSTR-1 (Structural Check under s.16(2)(aa))
MATCH (i:Invoice)-[:REPORTED_IN]->(g2:GSTR {type:'GSTR-2B'})
WHERE NOT EXISTS {
    MATCH (i)-[:REPORTED_IN]->(g1:GSTR {type:'GSTR-1'})
    WHERE g1.period = g2.period
}
RETURN i.id AS invoice_id, i.total_tax AS tax;

// 2. Multi-Hop Tax Payment Chain (Unpaid Tax under s.16(2)(c))
MATCH (v:Vendor)-[:ISSUED_INVOICE]->(i:Invoice)-[:REPORTED_IN]->(g:GSTR {type:'GSTR-1'})
MATCH (v)-[:FILED_RETURN]->(g3:GSTR3B {period: i.period})
WHERE g3.filed = false
RETURN i.id AS invoice_id, v.name AS vendor_name, g3.status AS status;
```

5. Predictive Vendor Compliance ML Model

To move from reactive audits to proactive risk prevention, the system includes a **Random Forest Classifier** (300 trees, depth 8, balanced class weights) served directly via FastAPI (risk_model.py).

8 Graph-Derived Feature Dimensions:

Feature Name	Description & Extraction Source
mismatch_count	Count of past mismatched invoices issued by vendor.
total_tax_at_risk	Total monetary tax value in disputed invoices (INR).
filing_delay_days	Average delay in days for return submissions.
graph_centrality	PageRank degree centrality score in the supply graph.
transaction_volume	Total transaction history volume.
community_cluster	Network community detection ID / State cluster.
einvoice_compliance_rate	Proportion of vendor invoices carrying valid IRN.
state_risk_factor	Historical state-level prior compliance risk score.

Avoiding the Bayes Error Trap: Synthetic training data uses a *stochastic logistic process* with Bernoulli sampling rather than a deterministic threshold. This introduces realistic noise, producing calibrated metrics: **Held-out Accuracy: 76.8%, ROC-AUC: 0.853, 5-Fold CV: 76.8%.**

6. Explainable Audit Trail Engine

Audit trails (audit_trail.py) are built from live graph facts rather than static hand-written text strings. Every audit card maps to statutory provisions under Indian Tax Law:

Mismatch Category	CGST Act Provision	Statutory Legal Explanation & Action Plan
Missing in GSTR-1	Section 16(2)(aa)	ITC restricted unless invoice is reported by supplier in GSTR-1 and appears in GSTR-2B. Withhold credit and request Table 9A amendment.
GSTR-3B Not Filed	Section 16(2)(c)	Credit is illegal unless tax collected by supplier was actually remitted to government. Do not utilize credit until return is filed.
Not in Purchase Reg.	Section 35(1)	Requirement to maintain true/correct books of accounts. Confirm goods receipt to guard against fake invoice scams.

E-Way Bill Missing	Rule 138	Consignments above ₹50,000 require proof of physical movement via an e-Way Bill.
e-Invoice Missing	Rule 48(4)	Notified suppliers must issue IRN. Invoice lacking IRN is not a valid tax invoice.

7. Subsystem Resilience & API Endpoints

The system degrades gracefully across all layers if any component is offline:

Subsystem	Primary Provider	Fallback Mechanism	User Impact
Database	MongoDB Atlas	Bundled mock dataset (mockData.js)	Zero downtime; offline banner displayed.
Knowledge Graph	Neo4j 5 Server	Client-side graph projection (DataContext)	Force graph still renders; engine reports 'client'.
Risk AI Model	RandomForest PKL	Weighted-sum heuristic algorithm	Risk scores continue generating smoothly.

Core REST API Endpoints Summary:

- GET /api/vendors & POST /api/vendors — Vendor CRUD with real-time ML risk prediction.
- GET /api/invoices & POST /api/invoices — Invoice CRUD with auto mismatch classification.
- POST /api/predict-risk — Scores vendor feature vectors using the Random Forest model.
- GET /api/reconcile — Runs graph-traversal reconciliation across specified filing periods.
- GET /api/audit-trail/{id} — Generates legally grounded explainable audit reports from graph facts.
- POST /api/graph/sync — Projects MongoDB collections into Neo4j nodes and edges.

Report Summary: This technical report documents the complete architectural, database, machine learning, and legal design of GST ReconcileAI. All systems have been verified operational.